

Overview

Version control is essential for AI engineering because teams iterate quickly across notebooks, datasets, pipelines, and model code. Git provides collaboration safety, reproducibility, and an audit trail of changes.

Core Git Concepts

- **Repository:** The project history and working files tracked by Git.
- **Commit:** A snapshot with a message describing what changed.
- **Branch:** An isolated line of development for a feature/fix/experiment.
- **Tag:** A named reference, often used to mark releases or milestones.

Branching & Workflow

A practical workflow: - Keep **main** stable. - Create feature branches for each task. - Open PRs to merge reviewed changes.

Brief comparison: - **GitFlow:** formal branch types (feature/release/hotfix), useful for structured release cycles. - **Trunk-based:** short-lived branches and frequent merges, useful for fast iteration teams.

Pull Requests & Code Review

PRs package changes for review before merging. Reviewers check correctness, maintainability, and potential regressions. Approvals improve quality and shared ownership.

Practical AI Engineering Scenarios

- Use experiment branches like **feature/experiment-new-feature** for model iterations.
- Use PRs to review notebook updates, data pipeline changes, and inference logic before production rollout.

Common Pitfalls

- Committing large data files directly to Git, causing slow clones and repository bloat.
- Mixing unrelated experiments in one branch, making review and rollback difficult.

Interview Prep Checklist

- Know core commands: **clone, status, commit, push, pull, merge, rebase**.
- Explain a branching strategy that keeps **main** stable while allowing experimentation.
- Explain how PR reviews reduce production risk in ML/AI projects.